🚀 OBJECT ORIENTED SYSTEMS (OOS) — 2022 QUESTION 1 COMPLETE SOLUTION

🌟 Introduction

This section contains the complete detailed solution of Question 1 (CO1) from the 2022 Object Oriented Systems examination paper 📘🧿. The answers are developed using the uploaded semester paper and handwritten classroom notes, along with additional expert-level explanations to maximize conceptual clarity and exam performance.

This guide focuses on:

- 🧠 Deep Java understanding
- 💡 Conceptual reasoning
- 🚀 Object-Oriented Programming principles
- 📝 Detailed code-based explanations
- 🎯 University-topper answer presentation

📃 QUESTION 1(a)

❓ **Justify the truth or falsity of the following statements with supporting arguments and suitable code snippets wherever necessary.**

🔹 Statement 1

❓ **"A non-static member method of a class can access only the non-static member variables."**

❌ **FALSE**

✅ **Explanation** A non-static member method belongs to an object of the class. Since objects can access both instance members and class-level members, a non-static method can access:

- ✅ Static variables
- ✅ Non-static variables

However, a static method cannot directly access non-static members because static methods belong to the class itself and not to any particular object.

According to Java memory architecture:

- **Static members** → stored in class memory area
- **Non-static members** → stored separately for every object

Since objects already have access to class-level members, non-static methods can access both.

✅ **Example Program**

```java
class Demo {
    // Static variable
    static int count = 100;

    // Non-static variable
    int value = 50;

    // Non-static method
    void show() {
        // Accessing non-static variable
        System.out.println("Value = " + value);
        // Accessing static variable
        System.out.println("Count = " + count);
    }

    public static void main(String[] args) {
        Demo obj = new Demo();
        obj.show();
    }
}
```

🖥️ **Output**

```
Value = 50
Count = 100
```

🎯 **Conclusion** Thus, the statement is false because non-static methods can access BOTH static and non-static variables 🚀.

🔹 Statement 2

❓ **"A static block of a class is executed every time an object of the class is created."**

❌ **FALSE**

✅ **Explanation** A `static block` executes ONLY ONCE when the class is loaded into memory by the JVM. It does NOT execute every time an object is created.

According to the handwritten notes:

- Static block executes once during class loading.
- Constructors execute every time objects are created, but static blocks execute only once regardless of the number of objects.

✅ **Example Program**

```
class Sample {
    // Static block
    static {
        System.out.println("Static Block Executed");
    }

    // Constructor
    Sample() {
        System.out.println("Constructor Executed");
    }

    public static void main(String[] args) {
        Sample s1 = new Sample();
        Sample s2 = new Sample();
        Sample s3 = new Sample();
    }
}
```

🖥️ **Output**

```
Static Block Executed
Constructor Executed
Constructor Executed
Constructor Executed
```

🎯 **Conclusion** The static block executes only once irrespective of how many objects are created 🚀.

🔹 Statement 3

❓ **"The main() method of a class cannot be overloaded."**

❌ **FALSE**

✅ **Explanation** The `main()` method CAN absolutely be overloaded in Java. However, JVM recognizes ONLY the following method as the program entry point: `public static void main(String args[])`

Other overloaded versions behave like normal methods and must be invoked manually.

✅ **Example Program**

```
class MainDemo {
    // Original main method
    public static void main(String[] args) {
        System.out.println("Original main method");
        main(10);
        main("OOS");
    }
```

```
    // Overloaded version
    public static void main(int x) {
        System.out.println("main(int) called");
    }

    // Overloaded version
    public static void main(String s) {
        System.out.println("main(String) called");
    }
}
```

🖥️ **Output**

```
Original main method
main(int) called
main(String) called
```

🎯 **Conclusion** Thus, the statement is false because the `main()` method CAN be overloaded 🚀.

🔹 Statement 4

❓ **"Arrays are reference data types in Java whereas int, float, char are primitive datatypes."**

✅ **TRUE**

✅ **Explanation** Primitive datatypes store actual values directly inside memory. Examples:

- `int`
- `float`
- `char`
- `boolean`

Arrays are different because array variables store references (memory addresses) pointing to array objects. Thus arrays are classified as `Reference Datatypes`.

✅ **Example Program**

```
class ArrayDemo {
    public static void main(String[] args) {
        int x = 10;
        int arr[] = {1, 2, 3};

        System.out.println(x);
        System.out.println(arr);
    }
}
```

🖥️ **Output**

```
10
[I@15db9742
```

💡 **Explanation** The strange array output proves that the variable stores a reference rather than raw values.

🎯 **Conclusion** Thus the statement is true because arrays are reference types while int, float, and char are primitive datatypes 🚀.

🔹 Statement 5

❓ **"An initialization block of a class can initialize both static and non-static variables of that class."**

✅ **TRUE**

✅ **Explanation** Initialization blocks execute before constructors. They can initialize:

- ✅ Static variables
- ✅ Non-static variables

Initialization blocks are useful for common initialization logic shared by all constructors.

✅ **Example Program**

```
class InitBlock {
    static int a;
    int b;

    // Initialization block
    {
        a = 100;
        b = 200;
    }

    void show() {
        System.out.println("a = " + a);
        System.out.println("b = " + b);
    }

    public static void main(String[] args) {
        InitBlock obj = new InitBlock();
        obj.show();
    }
}
```

🖥️ **Output**

```
a = 100
b = 200
```

🎯 **Conclusion** Thus initialization blocks can initialize both static and non-static variables 🚀.

🔹 Statement 6

❓ **"A class and its object occupy same amount of spaces in memory."**

❌ **FALSE**

✅ **Explanation** A class stores:

- Method definitions
- Static variables
- Metadata

An object stores:

- Instance variables
- Object-specific data

Different objects may consume different memory depending on instance data. Hence a class and its object cannot occupy equal memory.

✅ **Example Program**

```
class Student {
    int roll;
    String name;
}

class Demo {
    public static void main(String[] args) {
        Student s1 = new Student();
        Student s2 = new Student();

        System.out.println(s1);
        System.out.println(s2);
    }
}
```

💡 **Explanation** Each object receives separate memory allocation in heap memory.

🎯 **Conclusion** Thus the statement is false because class memory and object memory are fundamentally different 🚀.

📄 QUESTION 1(b)

❓ **State with valid reasons which pair of methods among the following can be overloaded.**

🔹 Pair 1

`public int sum(int, int)` `public static int sum(int, int)`

❌ **Cannot Be Overloaded**

✅ **Reason** The `static` keyword is NOT part of the method signature. Both methods effectively become `sum(int, int)`. Hence ambiguity occurs.

🔹 Pair 2

`public void sum(int, int)` `public void sum(int, int, int)`

✅ **Can Be Overloaded**

✅ **Reason** Number of parameters differs. Thus method signatures differ.

✅ **Example Program**

```
class Demo {
    void sum(int a, int b) {
        System.out.println(a + b);
    }
    void sum(int a, int b, int c) {
        System.out.println(a + b + c);
    }
}
```

🔹 Pair 3

`public int sum(int, int)` `public void sum(int, int)`

❌ **Cannot Be Overloaded**

✅ **Reason** Return type alone cannot distinguish overloaded methods. Method overloading depends ONLY on parameter list differences.

🔹 Pair 4

`public double sum(float, double)` `public float sum(double, float)`

✅ **Can Be Overloaded**

✅ **Reason** Parameter datatype and order differ. Hence method signatures are different.

✅ **Example Program**

```
class Demo {
    double sum(float a, double b) {
        return a + b;
    }
    float sum(double a, float b) {
        return (float)(a + b);
    }

    public static void main(String[] args) {
        Demo d = new Demo();
        System.out.println(d.sum(2.5f, 3.5));
        System.out.println(d.sum(5.5, 2.5f));
    }
}
```

🎯 **Conclusion** Method overloading depends ONLY on:

1. Number of parameters
2. Datatype of parameters
3. Order of parameters

Return type and static keyword do not participate in overloading 🚀.

📄 QUESTION 1(c)

❓ **Show how several member methods of a class can be invoked in cascaded fashion.**

✅ **Answer** `Cascaded method calling` means invoking multiple methods continuously using a single statement. This is achieved by returning `this` from methods.

🌟 **Advantages of Cascading** Method cascading:

- Improves readability
- Reduces repeated object names
- Produces elegant code
- Supports fluent programming style

✅ **Example Program**

```
class Student {
    int roll;
    String name;

    Student setRoll(int r) {
        roll = r;
        return this;
    }

    Student setName(String n) {
```

```
        name = n;
        return this;
    }

    Student show() {
        System.out.println("Roll = " + roll);
        System.out.println("Name = " + name);
        return this;
    }
}

class Demo {
    public static void main(String[] args) {
        Student s = new Student();

        s.setRoll(101)
         .setName("Arpan")
         .show();
    }
}
```

🖥️ **Output**

```
Roll = 101
Name = Arpan
```

💡 **Explanation** Each method returns `this` which represents the current object reference. Thus the next method can immediately be invoked.

🎯 **Conclusion** Cascading method calls produce concise and elegant Object-Oriented code 🚀.

📄 QUESTION 1(d)

❓ **What do you mean by anonymous object in Java? Illustrate cascading method call in relation to it.**

✅ **Answer** An `anonymous object` is an object created without any reference variable.

Example: `new Demo();` The object exists temporarily and is usually used only once.

🌟 **Advantages of Anonymous Objects** Anonymous objects:

- Reduce unnecessary references
- Save memory references
- Produce compact code
- Useful for one-time operations

✅ **Example Program**

```
class Sample {
    Sample show() {
        System.out.println("Show Method");
        return this;
    }

    Sample display() {
        System.out.println("Display Method");
        return this;
    }
}

class Demo {
    public static void main(String[] args) {
        new Sample().show().display();
    }
}
```

🖥️ **Output**

```
Show Method
Display Method
```

💡 **Explanation** Here: `new Sample()` creates an anonymous object. Method calls are cascaded using returned object references.

🎯 **Conclusion** Anonymous objects combined with cascading produce elegant and compact Java programs 🚀.

📃 QUESTION 1(e)

❓ **Consider a StringBuffer object created as follows. Discuss the output after each statement.**

```
StringBuffer s = new StringBuffer("JU IT OOS");
s.ensureCapacity(30);
System.out.println("Capacity="+s.capacity()+ " Length="+s.length());
s.append("2nd year");
s.insert(2, "2nd sem");
s.replace(5, 8, "Java");
```

✅ **Answer** `StringBuffer` is a mutable sequence of characters. Unlike `String`, `StringBuffer` objects can be modified directly without creating new objects. According to the notes: StringBuffer is mutable whereas String is immutable.

🌟 **Initial String** `JU IT OOS` Length: `9`

🔹 **Step 1** `s.ensureCapacity(30);` This ensures minimum capacity = 30.

🔹 **Step 2** `System.out.println("Capacity="+s.capacity()+" Length="+s.length());` 💡 **Capacity Calculation** Default capacity: `Initial Length + 16` Initial string length: `9` Thus: `9 + 16 = 25` After: `ensureCapacity(30)` new capacity becomes: `(25 × 2) + 2 = 52`

🖥️ **Output**

```
Capacity=52 Length=9
```

🔹 **Step 3** `s.append("2nd year");` New string becomes: `JU IT OOS2nd year`

🔹 **Step 4** `s.insert(2, "2nd sem");` New string becomes: `JU2nd sem IT OOS2nd year`

🔹 **Step 5** `s.replace(5, 8, "Java");` Characters from index 5 to 7 are replaced. Final string becomes: `JU2ndJavam IT OOS2nd year`

✅ **Complete Program**

```
class StringBufferDemo {
    public static void main(String[] args) {
        StringBuffer s = new StringBuffer("JU IT OOS");

        s.ensureCapacity(30);
        System.out.println("Capacity = " + s.capacity());
        System.out.println("Length = " + s.length());

        s.append("2nd year");
        System.out.println(s);

        s.insert(2, "2nd sem");
        System.out.println(s);

        s.replace(5, 8, "Java");
        System.out.println(s);
    }
}
```

🎯 **Conclusion** `StringBuffer` provides mutable string operations like:

- `append()`
- `insert()`
- `replace()`
- `ensureCapacity()` making it highly efficient for repeated string modification 🚀.